一、思考题

Thinking 0.1 思考下列有关 Git 的问题：

- 在前述已初始化的 `~/learnGit` 目录下，创建一个名为 `README.txt` 的文件。执行命令 `git status > Untracked.txt`。
- 在 `README.txt` 文件中添加任意文件内容，然后使用 `add` 命令，再执行命令 `git status > Stage.txt`。
- 提交 `README.txt`，并在提交说明里写入自己的学号。
- 执行命令 `cat Untracked.txt` 和 `cat Stage.txt`，对比两次运行的结果，体会 `README.txt` 两次所处位置的不同。
- 修改 `README.txt` 文件，再执行命令 `git status > Modified.txt`。
- 执行命令 `cat Modified.txt`，观察其结果和第一次执行 `add` 命令之前的 status 是否一样，并思考原因。

不一样。第一次add之前，文件处于 untracked 状态，进行了add后，文件进入暂存区，此时处于stage状态，commit后进入版本库。在第二次修改了文件内容后，修改的是工作区的内容，暂存区的文件内容未被修改，文件处于modified状态，此时要用add指令将暂存区文件修改为stage状态。

Thinking 0.2 仔细看看0.10，思考一下箭头中的 add the file 、stage the file 和 commit 分别对应的是 Git 里的哪些命令呢？

add the file对应的是git add指令，表示跟踪文件。stage the file对应的是git add指令，表示修改后加入暂存区。commit对应的是git commit -m "<message>"。

Thinking 0.3 思考下列问题：

1. 代码文件 `print.c` 被错误删除时，应当使用什么命令将其恢复？
2. 代码文件 `print.c` 被错误删除后，执行了 `git rm print.c` 命令，此时应当使用什么命令将其恢复？
3. 无关文件 `hello.txt` 已经被添加到暂存区时，如何在不删除此文件的前提下将其移出暂存区？

1. 使用git checkout -- printf.c从暂存区中找回。
2. 首先使用git reset HEAD printf.c重写暂存区的目录树，然后再使用git checkout -- printf.c从暂存区找回文件。
3. 使用git rm --cached printf.c直接从暂存区删除文件，但是工作区保持不变。

Thinking 0.4 思考下列有关 Git 的问题：

- 找到在 `/home/22xxxxxx/learnGit` 下刚刚创建的 `README.txt` 文件，若不存在则新建该文件。
- 在文件里加入 `Testing 1`，`git add`，`git commit`，提交说明记为 1。
- 模仿上述做法，把 1 分别改为 2 和 3，再提交两次。
- 使用 `git log` 命令查看提交日志，看是否已经有三次提交，记下提交说明为 3 的哈希值[a]。
- 进行版本回退。执行命令 `git reset --hard HEAD^` 后，再执行 `git log`，观察其变化。
- 找到提交说明为 1 的哈希值，执行命令 `git reset --hard <hash>` 后，再执行 `git log`，观察其变化。
- 现在已经回到了旧版本，为了再次回到新版本，执行 `git reset --hard <hash>`，再执行 `git log`，观察其变化。

[a]使用 `git log` 命令时，在 `commit` 标识符后的一长串数字和字母组成的字符串

Git版本库记录着每一次提交的版本，提交的哈希值为最近一次提交的哈希值。如用提交说明为1的哈希值，使用git reset，则会回到提交版本1和版本1以前的版本。如用提交说明为3的哈希值，则会到提交说明为3的版本，该版本以前的提交会在git log中。

Thinking 0.5 执行如下命令，并查看结果

- `echo first`
- `echo second > output.txt`
- `echo third > output.txt`
- `echo forth >> output.txt`

执行echo first时，直接在当前窗口输出了first。执行echo second > output.txt后，在output.txt文件中输出了second。执行echo third > output.txt后，在output.txt文件

中输出了third，之前输出的second被覆盖掉了。执行echo forth >> output.txt后，在output.txt文件中输出了forth，且之前输出的third并没有被覆盖。

> Thinking 0.6 使用你知道的方法（包括重定向）创建下图内容的文件（文件命名为 test），将创建该文件的命令序列保存在 command 文件中，并将 test 文件作为批处理文件运行，将运行结果输出至 result 文件中。给出 command 文件和 result 文件的内容，并对最后的结果进行解释说明（可以从 test 文件的内容入手）。具体实现的过程中思考下列问题：echo echo Shell Start 与 echo `echo Shell Start`效果是否有区别；echo echo $c>file1 与 echo `echo $c>file1`效果是否有区别。

command文件中的代码如下：

```
echo echo Shell Start... > test
echo echo set a = 1 >> test
echo a=1 >> test
echo echo set b = 2 >> test
echo b=2 >> test
echo echo set c = a+b >> test
echo c=\$[\$a+\$b] >> test
echo echo c = \$c >> test
echo echo save c to ./file1 >> test
echo echo \$c\>file1 >> test
echo echo save b to ./file2 >> test
echo echo \$b\>file2 >> test
echo echo save a to ./file3 >> test
echo echo \$a\>file3 >> test
echo echo save file1 file2 file3 to file4 >> test
echo cat file1\>file4 >> test
echo cat file2\>\>file4 >> test
echo cat file3\>\>file4 >> test
echo echo save file4 to ./result >> test
echo cat file4\>\>result >> test
```

echo echo Shell Start和echo ‘echo Shell Start’在效果上有区别。

echo echo Shell Start是只出入echo Shell Start。

echo ‘echo Shell Start’是出入执行echo Shell Start的结果。

echo echo $c>file1’和echo ‘echo $c>file1’的区别为

echo echo $c>file1是出入c的字符串

echo ‘echo $c>file1‘是把字符串c来写file1。

二、难点分析

本次实验遇到的最难点在Exercise0.4的第二小题。这是看到完成两种Makefile的问题后,很难轻易接近的问题。 后来发现需要使用外部Makefile呼叫内部Makefile，同时理

解了问题。其他难点包括exercise0.3题和0.4小题。虽然结果都很准确，但测试仪一直显示检查结果说没有找到目录或文件，虽然解题是对的，但遗憾的是没能得分。

三、实验体会

因为是第一次接触Linux本身，所以阅读和理解指南花费了很长时间，现在还处于不成熟的状态。 希望能多练习，提高熟练度。 虽然此次lab0实验的难度不高，但即将进行的实验难度非常大。 而且还要培养在输入指令失误时恢复的能力。